Nested Loops

```
                    ; Opcode:
    mov  ecx, 5     ; B9 05 00 00 00
outer:
    push ecx        ; 51
    :::
    mov  ecx, 10    ; B9 0A 00 00 00
inner:
    push ecx        ; 51
    :::
    pop  ecx        ; 59
    loop inner      ; E2 FC(-4)
    :::
    pop  ecx        ; 59
    loop outer      ; E2 F3(-13)
```

Repeats 5 times

Repeats 5 * 10 times

Analog in HLL, e.g. C:

```
for (size_t r = 5; r > 0; r--)
{ ::: outer loop
  for (size_t c = 10; c > 0; c--)
  { ::: inner loop
  }
}
```

Nested looping example 1: Traversing matrix elements (Row-by-Row)

```
; :::
.CONST
; Input matrix
M   DD 1,0,9,8,0,7,8,0
    DD 1,0,9,8,0,7,8,0
    DD 1,0,9,8,0,7,8,0
    DD 1,0,9,8,0,7,8,0
    DD 1,0,9,8,0,7,8,0
; Input array M dimensions
ROWS EQU 5
COLS EQU 8
.DATA?
ZC  DD      ? ; Result variable - the number of zeroes in M

.CODE
    ; :::
    lea     ebx, M
    xor     eax, eax
    mov     ecx, ROWS
Row_loop:
    push    ecx         ; save rows counter
    xor     esi, esi    ; reset column offset for the current row
    mov     ecx, COLS
Col_loop:
    cmp     dword ptr [ebx + esi], 0
    jne     Skip
    inc     eax
Skip:
    add     esi, 4
    loop    Col_loop
    add     ebx, esi    ; move to the next row offset
    pop     ecx         ; restore rows counter
    loop    Row_loop
    mov     [ZC], eax   ; store the result
    ; :::
```

Nested looping example 2:
Count negatives for each column (Column-by-Column)

```
; :::
.CONST
; Input matrix
M   DD 1,0,9,8,0,7,8,0
   ;DD ::: - seven lines of data
; Input array M dimensions
ROWS EQU 5
COLS EQU 8
.DATA?
R  DD COLS DUP(?) ; The result array
.CODE
; :::
    xor     esi, esi            ; current column
    xor     edi, edi            ; index in the array R
    mov     ecx, COLS           ; number of columns
Col_loop:
    push    ecx
    lea     ebx, M
    push    esi
    xor     eax, eax            ; counter for col == 0
    mov     ecx, ROWS           ; number of rows
Row_loop:
    cmp dword ptr [ebx + esi*4], 0 ; check the value of current element
    jge     Skip
    inc     eax                 ; found new negative element in current column
Skip:
    add     esi, COLS           ; go to the next row
    loop    Row_loop
    lea     ebx, R
    mov     [ebx + edi*4], eax  ; save count into the array result[]
    pop     esi
    inc     esi                 ; next column
    inc     edi                 ; next index in the array result[]
    pop     ecx
    loop    Col_loop
    ; :::
```